

SMART WILD PIG DETECTION AND CROP PROTECTION USING IoT AND AI

Phase4- Prototype Development Review (PDR), Deliverables

TEAM MEMBERS:

DHANYA SRI J

LAVANYA V

ARUNACHALAM M

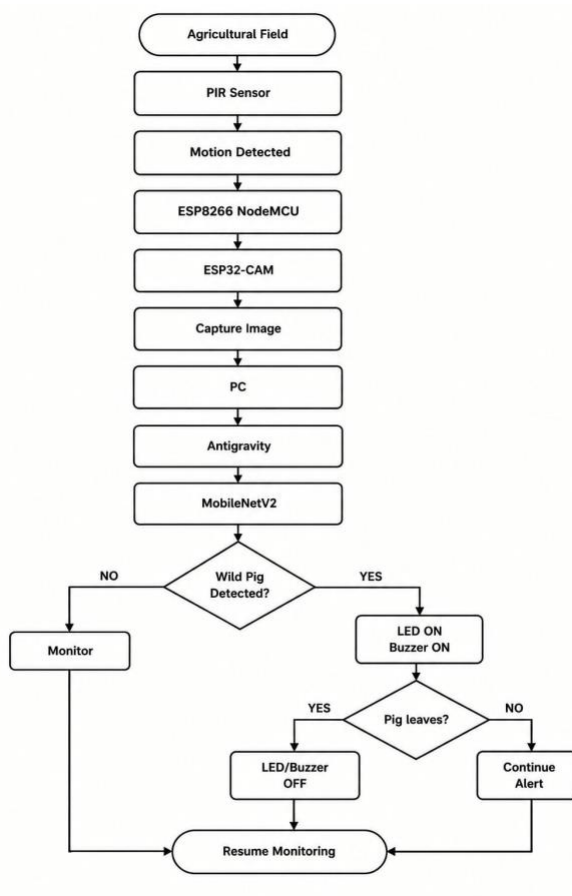
SUJITH N

Working Prototype :

The Version 1 prototype consists of:

- PIR sensor

- ESP8266 NodeMCU
- ESP32-CAM
- LED
- Buzzer
- Computer/PC
- Arduino IDE
- Antigravity development environment
- MobileNetV2 computer-vision model



The Version 1 prototype demonstrates the basic integration of sensing, image acquisition, AI-based classification, and alert generation.

Embedded Software:

The embedded software is developed using the Arduino IDE.

ESP8266 NodeMCU:

The ESP8266 is responsible for:

- Receiving the output from the PIR sensor.
- Identifying a motion event.
- Controlling the trigger/communication sequence.
- Communicating with the ESP32-CAM as required by the prototype.
- Supporting the overall detection workflow.

ESP32-CAM:

The ESP32-CAM is responsible for:

- Receiving the trigger.
- Activating the camera.
- Capturing the image of the detected object/animal.
- Sending the captured image to the computer-processing stage.

LED and Buzzer:

The LED and buzzer are controlled as the local alert system.

- LED ON: Wild pig detection alert.
- Buzzer ON: Audible wild pig detection alert.
- LED/Buzzer OFF: No active detection condition.

PCB Fabrication Status:

Current Status: Prototype / Breadboard Implementation

For Version 1, the system can be implemented using:

- Breadboard
- Jumper wires
- Development boards
- Sensor modules
- LED
- Buzzer

A custom PCB is not required for Version 1.

Future PCB Development

PCB Fabrication Status: Not applicable for current prototype.

Mechanical Assembly:

The mechanical assembly involves physically mounting the components in suitable positions.

Camera Mounting:

The ESP32-CAM should be positioned so that:

- The agricultural field area is clearly visible.
- The camera has a suitable viewing angle.
- The camera is stable.
- The lens is not blocked.
- The camera is protected from accidental movement.

PIR Sensor Mounting:

The PIR sensor should be positioned to:

- Cover the required detection area.
- Detect movement approaching the camera's field of view.
- Reduce unnecessary detection from outside the target area.

Alert Components:

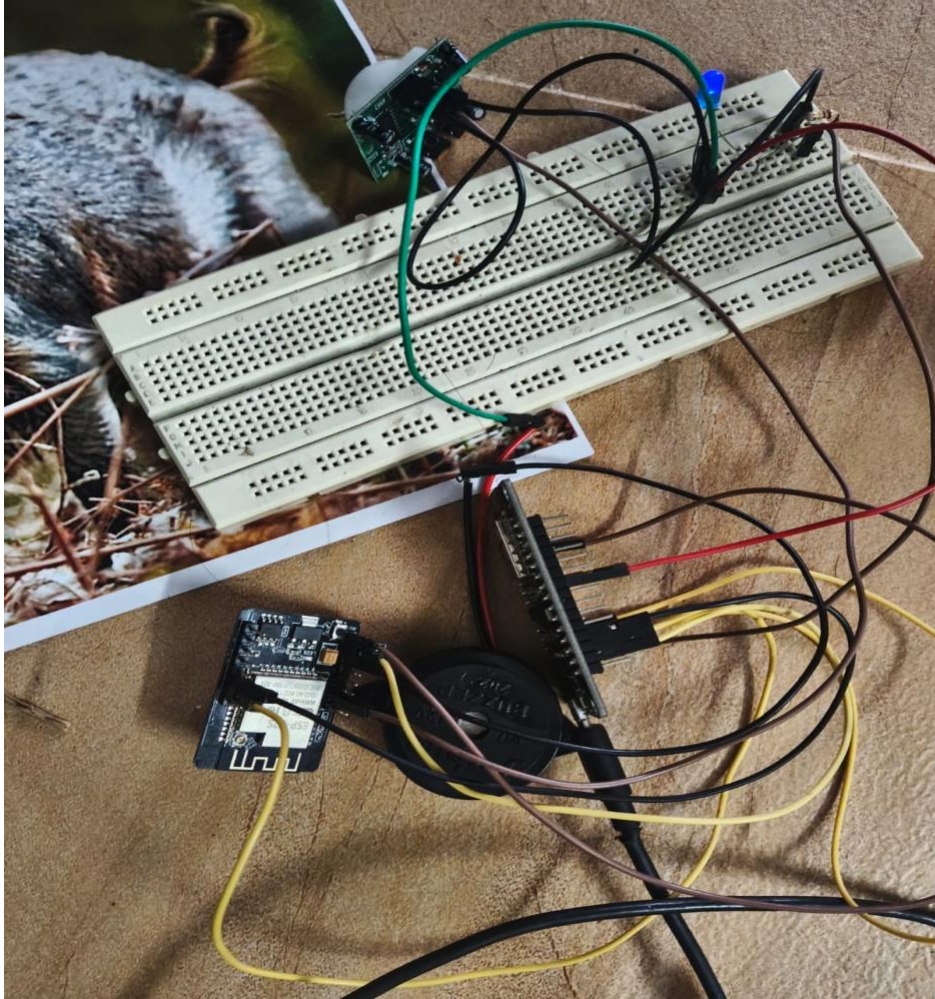
The LED and buzzer should be placed where they can be easily noticed or heard during testing.

For actual field deployment, all components should be placed inside a suitable protective enclosure.

Hardware Integration:

Hardware integration combines all the individual components into one working system.

S.No	Component	Purpose
1	ESP32-CAM	Image capture and Wi-Fi communication
2	PIR Sensor	Detects motion
3	LED	Visual alert
4	Passive Buzzer	Audible alert
5	NodeMCU ESP8266	USB-to-serial programming interface
6	Resistors	LED current limiting
7	Power Supply	Provides power to the circuit



The hardware integration is considered successful when the motion detected by the PIR sensor results in image capture and the appropriate alert is generated after AI-based verification.

Software Integration:

The software portion consists of embedded firmware and computer-based AI processing.

Software Components:

Arduino IDE

- Used to develop and upload ESP8266 and ESP32-CAM programs.

Antigravity

- Used as part of the development and computer-processing workflow.

MobileNetV2

- Used as the computer-vision model for image classification.

AI / Computer Vision Integration:

The captured image is processed using the MobileNetV2 computer-vision model.

The purpose of the AI stage is to reduce false alerts that may occur when the PIR sensor detects movement from humans, dogs, or other objects.

Example:

If the PIR sensor detects:

Human → Camera captures image → MobileNetV2 → Non-wild pig → No wild-pig alert

If the PIR sensor detects:

Wild pig → Camera captures image → MobileNetV2 → Wild pig → LED + Buzzer ON

This provides an additional verification stage instead of depending only on motion detection.

GitHub Repository:

https://github.com/dhanya-sri22/Smart_Wild_Pig_Detection.git

Weekly Progress Report:

Week 1 – Problem Identification:

- Identified the problem of wild pig attacks on agricultural crops.
- Studied existing animal detection methods.
- Defined the project objectives.
- Finalized the basic system concept.

Week 2 – Component Selection:

Selected the major components:

- PIR sensor
- ESP8266 NodeMCU
- ESP32-CAM
- LED
- Buzzer

Software tools were also selected.

Week 3 – Hardware Setup:

- Connected PIR sensor to ESP8266.
- Tested PIR motion detection.

- Connected ESP32-CAM.
- Tested camera operation.
- Tested LED and buzzer separately

Week 4 – Hardware Integration:

- Integrated PIR with ESP8266.
- Established the trigger mechanism for the ESP32-CAM.
- Tested image capture after motion detection.
- Checked communication between the hardware and PC.

Week 5 – AI Integration:

- Prepared wild pig and non-wild-pig images.
- Set up the MobileNetV2 model.
- Tested image classification.
- Checked detection results.

Week 6 – Alert Integration:

- Integrated the LED and buzzer.
- Tested alert activation after wild pig classification.
- Tested alert deactivation.
- Verified return to monitoring mode.

Week 7 – Prototype Testing:

- Conducted functional testing.
- Tested different motion conditions.
- Tested wild pig and non-wild-pig images.
- Identified errors and debugging requirements.

Prototype Demonstration:

The image is transferred to the computer.

Step 1

Power ON the prototype.

Step 2

The PIR sensor continuously monitors for movement.

Step 3

Movement is introduced in front of the PIR sensor.

Step 4

The ESP8266 receives the motion signal.

Step 5

The ESP32-CAM captures an image.

Step 6

When the detection condition is cleared:

LED → OFF

Buzzer → OFF

Step 7

The MobileNetV2 model analyzes the image.

Step 8

If the image is classified as a wild pig:

LED → ON

Buzzer → ON

Step 9

When the detection condition is cleared:

LED → OFF

Buzzer → OFF

Step 10

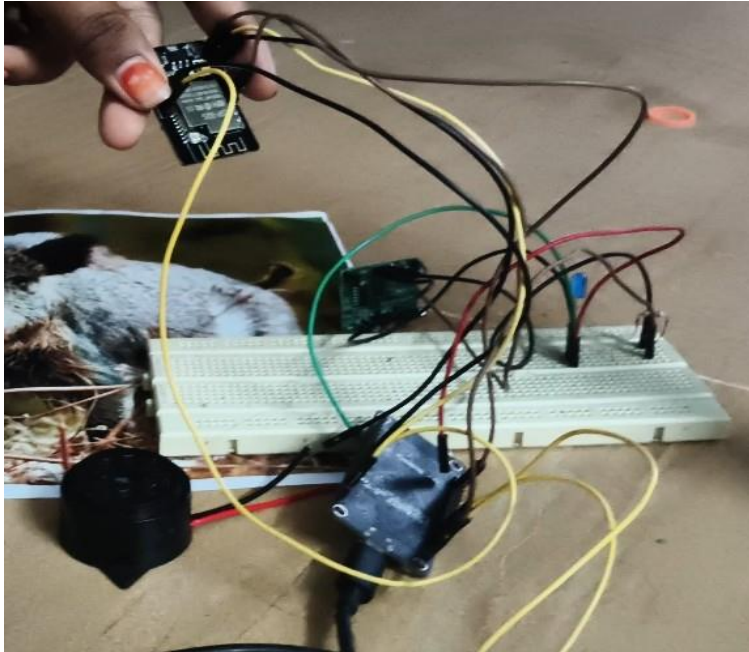
The system resumes monitoring.

SUBMIT:

Github Repository:

https://github.com/dhanya-sri22/Smart_Wild_Pig_Detection.git

Prototype Images:



Train model:

```
terminal  Help  smart_wild_pig_detection - Antigravity IDE - train_model.py  [Icons]
...  check_dataset.py  train_model.py 1 x  [Icons]

train_model.py > train_data
# image detection
6  IMG_SIZE = (224, 224)
7  BATCH_SIZE = 32
8
9  # Load dataset
10 train_data = tf.keras.utils.image_dataset_from_directory(
11     "train",
12     image_size=IMG_SIZE,
13
14     batch_size=BATCH_SIZE,
15     label_mode="binary"
16 )
17
18 valid_data = tf.keras.utils.image_dataset_from_directory(

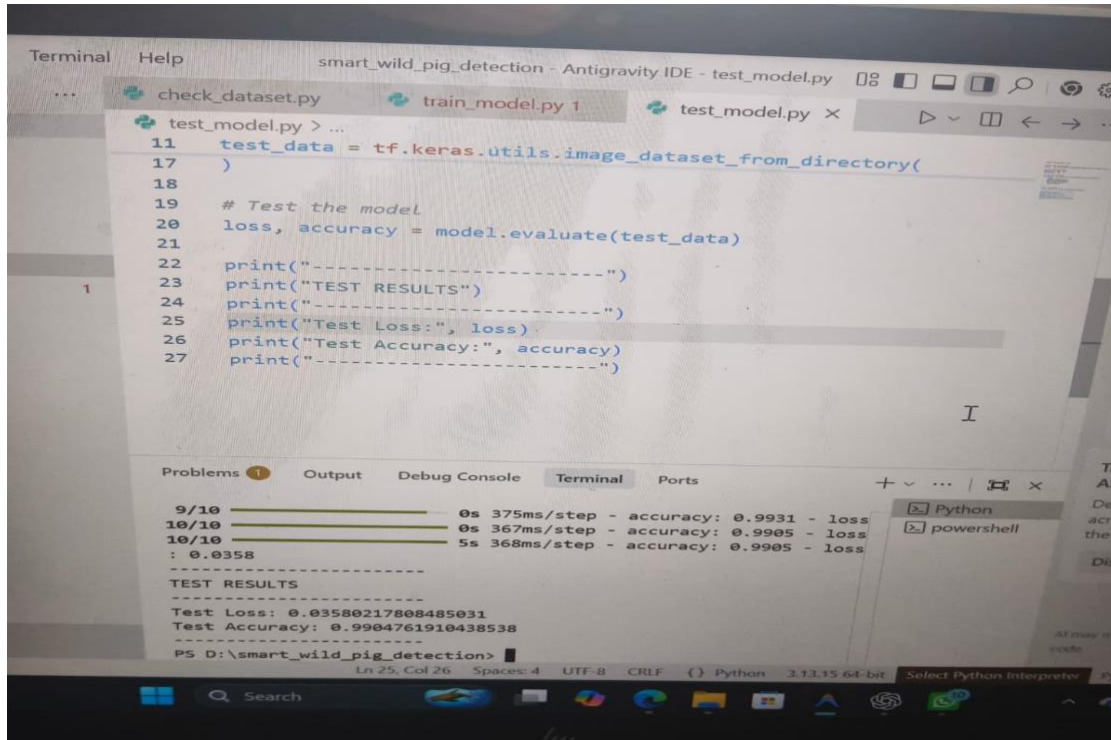
Problems 1  Output  Debug Console  Terminal  Ports  + v ... | [Icons]

38/47  [Progress Bar]  3s 399ms/step - accuracy: 0.9967 - loss
39/47  [Progress Bar]  3s 399ms/step - accuracy: 0.9968 - loss
40/47  [Progress Bar]  2s 400ms/step - accuracy: 0.9969 - loss
41/47  [Progress Bar]  2s 399ms/step - accuracy: 0.9970 - loss
42/47  [Progress Bar]  1s 399ms/step - accuracy: 0.9955 - loss
43/47  [Progress Bar]  1s 398ms/step - accuracy: 0.9956 - loss
44/47  [Progress Bar]  1s 398ms/step - accuracy: 0.9957 - loss
45/47  [Progress Bar]  0s 397ms/step - accuracy: 0.9958 - loss
46/47  [Progress Bar]  0s 397ms/step - accuracy: 0.9959 - loss
47/47  [Progress Bar]  0s 390ms/step - accuracy: 0.9959 - loss
47/47  [Progress Bar]  22s 467ms/step - accuracy: 0.9959 - los
s: 0.0177 - val_accuracy: 0.9894 - val_loss: 0.0411
Training completed!
Model saved as pig_detection_model.keras
PS D:\smart_wild_pig_detection>

Ln 12, Col 25  Spaces: 4  UTF-8  CRLF  {}  Python  3.13.15 64-bit  Select P

[Windows Taskbar Icons]
```

Test model:



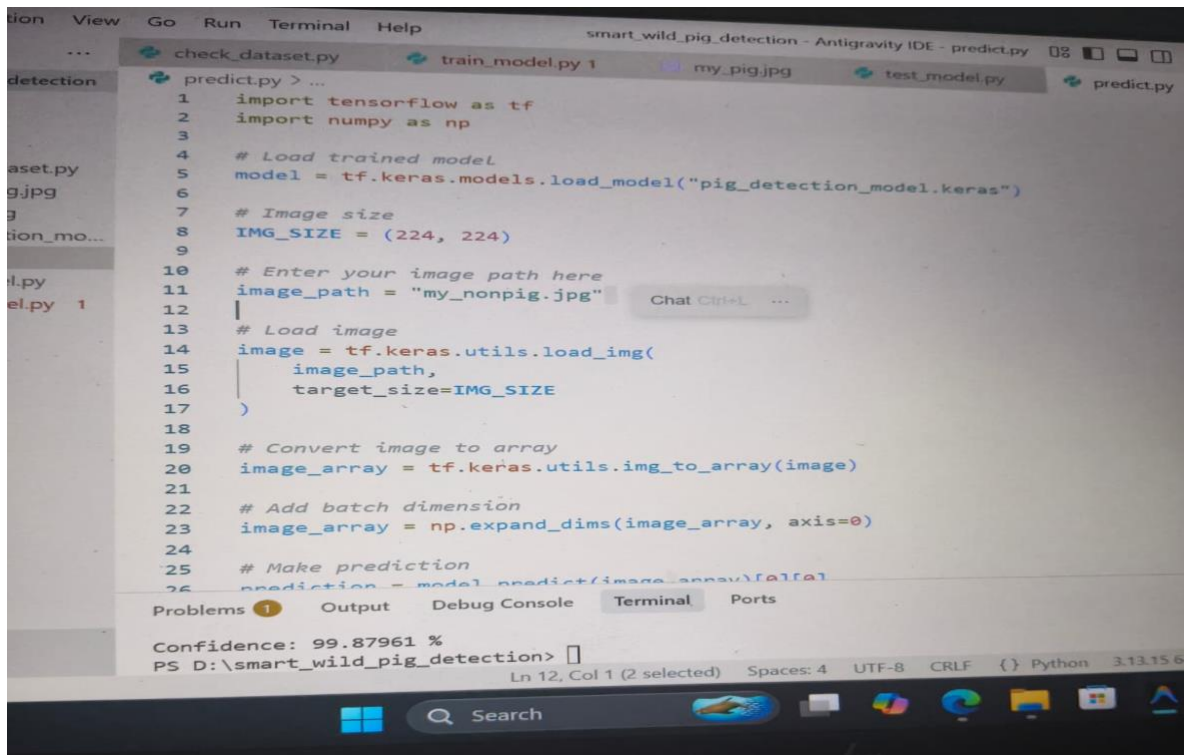
The screenshot shows the Antigravity IDE with the `test_model.py` file open. The script defines a test function that evaluates a trained model. The terminal output shows the results of the evaluation.

```
11 test_data = tf.keras.utils.image_dataset_from_directory(
17 )
18
19 # Test the model
20 loss, accuracy = model.evaluate(test_data)
21
22 print("-----")
23 print("TEST RESULTS")
24 print("-----")
25 print("Test Loss:", loss)
26 print("Test Accuracy:", accuracy)
27 print("-----")
```

Terminal Output:

```
9/10 ----- 0s 375ms/step - accuracy: 0.9931 - loss
10/10 ----- 0s 367ms/step - accuracy: 0.9905 - loss
10/10 ----- 5s 368ms/step - accuracy: 0.9905 - loss
: 0.0358
-----
TEST RESULTS
-----
Test Loss: 0.03580217808485031
Test Accuracy: 0.9904761910438538
-----
PS D:\smart_wild_pig_detection>
```

Predict model:



The screenshot shows the Antigravity IDE with the `predict.py` file open. The script imports TensorFlow and NumPy, loads a trained model, and defines functions to load and preprocess an image for prediction.

```
1 import tensorflow as tf
2 import numpy as np
3
4 # Load trained model
5 model = tf.keras.models.load_model("pig_detection_model.keras")
6
7 # Image size
8 IMG_SIZE = (224, 224)
9
10 # Enter your image path here
11 image_path = "my_nonpig.jpg"
12
13 # Load image
14 image = tf.keras.utils.load_img(
15     image_path,
16     target_size=IMG_SIZE
17 )
18
19 # Convert image to array
20 image_array = tf.keras.utils.img_to_array(image)
21
22 # Add batch dimension
23 image_array = np.expand_dims(image_array, axis=0)
24
25 # Make prediction
26 prediction = model.predict(image_array)[0][0]
```

Source code:

Weekly Progress Report:

Week 1 – Problem Identification:

- Identified the problem of wild pig attacks on agricultural crops.
- Studied existing animal detection methods.
- Defined the project objectives.
- Finalized the basic system concept.

Week 2 – Component Selection:

Selected the major components:

- PIR sensor
- ESP8266 NodeMCU
- ESP32-CAM
- LED
- Buzzer

Software tools were also selected.

Week 3 – Hardware Setup:

- Connected PIR sensor to ESP8266.
- Tested PIR motion detection.
- Connected ESP32-CAM.
- Tested camera operation.
- Tested LED and buzzer separately

Week 4 – Hardware Integration:

- Integrated PIR with ESP8266.
- Established the trigger mechanism for the ESP32-CAM.
- Tested image capture after motion detection.

- Checked communication between the hardware and PC.

Week 5 – AI Integration:

- Prepared wild pig and non-wild-pig images.
- Set up the MobileNetV2 model.
- Tested image classification.
- Checked detection results.

Week 6 – Alert Integration:

- Integrated the LED and buzzer.
- Tested alert activation after wild pig classification.
- Tested alert deactivation.
- Verified return to monitoring mode.

Week 7 – Prototype Testing:

- Conducted functional testing.
- Tested different motion conditions.
- Tested wild pig and non-wild-pig images.
- Identified errors and debugging requirements.

